OOP Lab Tasks

Lab 5

Submitted by :

Omama Tanveer

Submitted to :

Sir Mubashir sb

Reg No:

2026-S-CYS-25

Task 1:

Create a class Rectangle with width and height, a constructor, and methods area(), perimeter() and isSquare(). Create two objects and print the one with the larger area.

```
class Rectangle {
    double width, height;

    Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    double area() {
        return width * height;
    }

    double perimeter() {
        return 2 * (width + height);
    }

    boolean isSquare() {
        return width == height;
    }
```

```
}

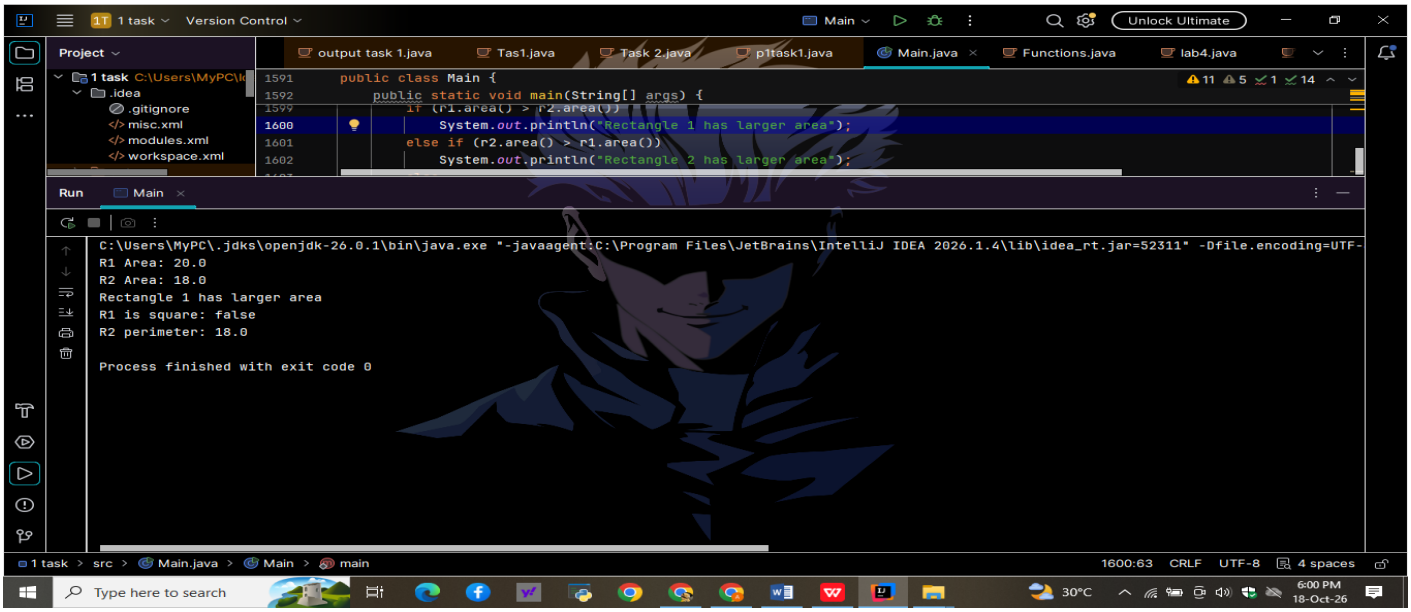
public class Main {
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle(5, 4);
        Rectangle r2 = new Rectangle(6, 3);

        System.out.println("R1 Area: " + r1.area());
        System.out.println("R2 Area: " + r2.area());

        if (r1.area() > r2.area())
            System.out.println("Rectangle 1 has larger area");
        else if (r2.area() > r1.area())
            System.out.println("Rectangle 2 has larger area");
        else
            System.out.println("Both have equal area");

        System.out.println("R1 is square: " + r1.isSquare());
        System.out.println("R2 perimeter: " + r2.perimeter());
    }
}
```

Output:



Task2:

Create a class Counter that tracks how many objects have been created (static) and how many times each object has been incremented (instance). Create 3 objects, increment them different numbers of times, and print both kinds of counts.

```
class Counter {
    static int totalObjects = 0;
    int count = 0;

    Counter() {
        totalObjects++;
    }

    void increment() {
        count++;
```

```
    }
}

public class Main {
    public static void main(String[] args) {
        Counter c1 = new Counter();
        Counter c2 = new Counter();
        Counter c3 = new Counter();

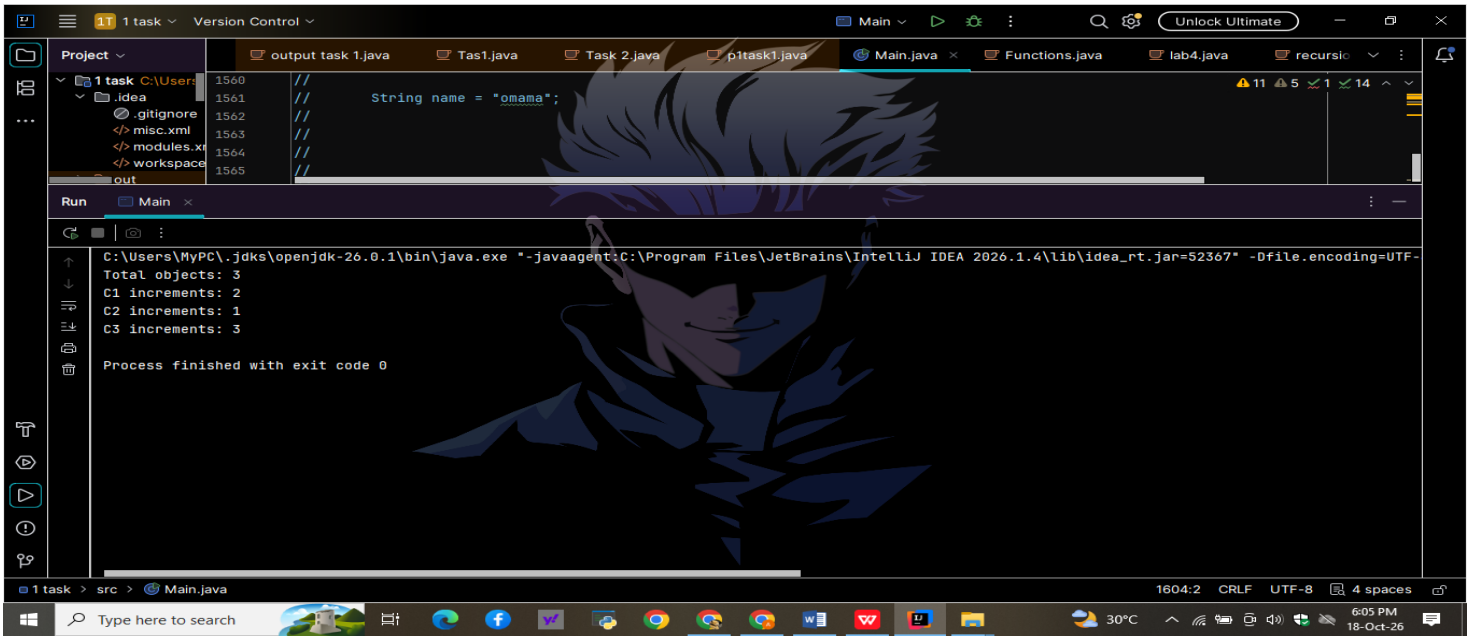
        c1.increment();
        c1.increment();

        c2.increment();

        c3.increment();
        c3.increment();
        c3.increment();

        System.out.println("Total objects: " + Counter.totalObjects);
        System.out.println("C1 increments: " + c1.count);
        System.out.println("C2 increments: " + c2.count);
        System.out.println("C3 increments: " + c3.count);
    }
}
```

Output:



Task 3:

Create a BankAccount class with a private balance and methods deposit(), withdraw() (reject overdrafts) and getBalance(). Simulate 5 mixed operations and print the balance after each.

```
class BankAccount {
    private double balance;

    BankAccount(double balance) {
        this.balance = balance;
    }

    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        } else {
```

```
            System.out.println("Invalid deposit");
        }
    }

    void withdraw(double amount) {
        if (amount <= 0) {
            System.out.println("Invalid amount");
        } else if (amount > balance) {
            System.out.println("Insufficient balance");
        } else {
            balance -= amount;
        }
    }

    double getBalance() {
        return balance;
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount(1000);

        account.deposit(500);
        System.out.println(account.getBalance());

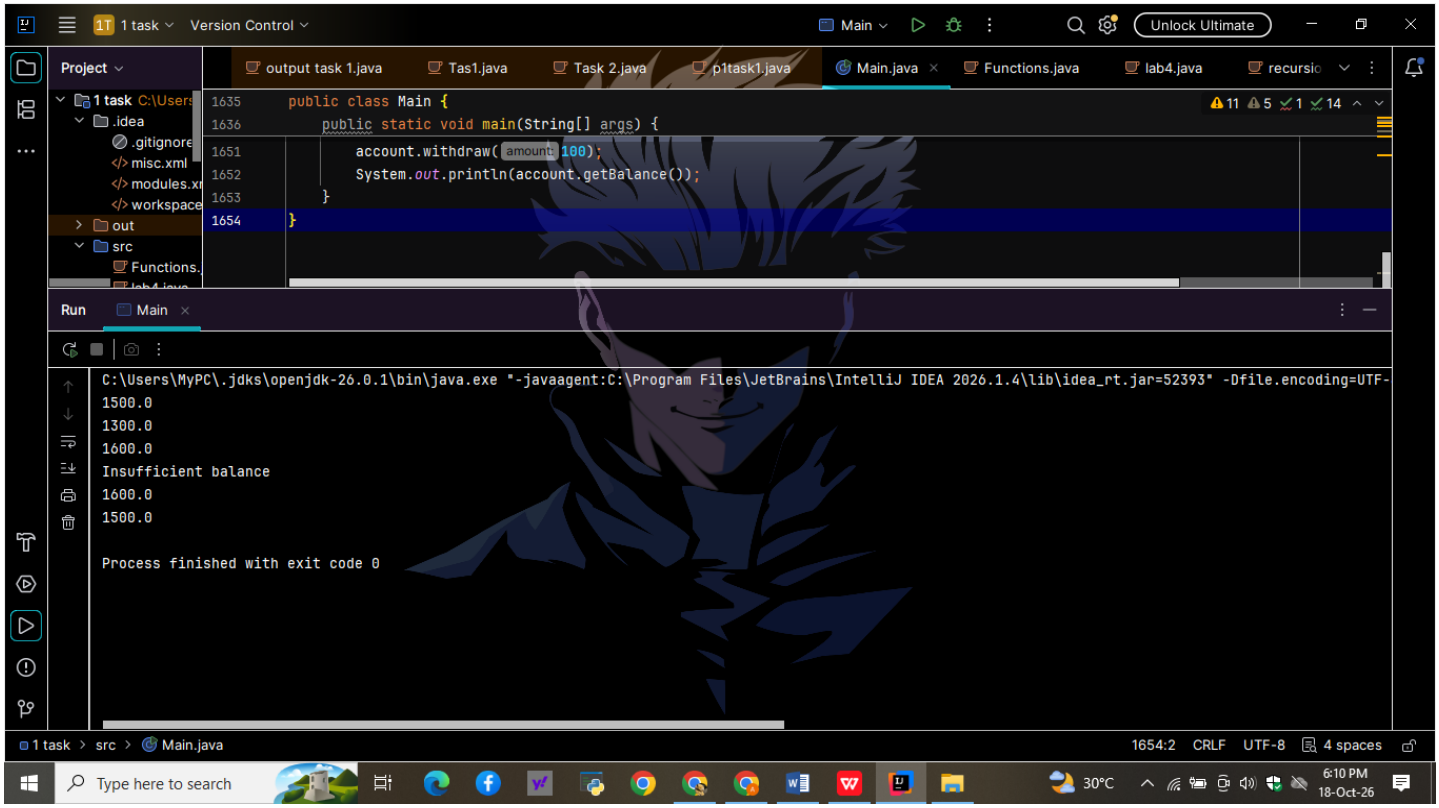
        account.withdraw(200);
        System.out.println(account.getBalance());
```

```
        account.deposit(300);
        System.out.println(account.getBalance());

        account.withdraw(2000);
        System.out.println(account.getBalance());

        account.withdraw(100);
        System.out.println(account.getBalance());
    }
}
```

Output:



Task 4:

Create a Student class with a name and an array of marks, and methods average() and grade(). Store 3 students in an array and print the topper.

```
class Student {
    String name;
    int[] marks;

    Student(String name, int[] marks) {
        this.name = name;
        this.marks = marks;
    }

    double average() {
        int sum = 0;

        for (int mark : marks) {
            sum += mark;
        }

        return (double) sum / marks.length;
    }

    String grade() {
        double avg = average();

        if (avg >= 80) return "A";
        if (avg >= 70) return "B";
        if (avg >= 60) return "C";
        if (avg >= 50) return "D";
```

```
        return "F";
    }
}

public class Main {
    public static void main(String[] args) {
        Student[] students = {
            new Student("Ali", new int[]{80, 90, 85}),
            new Student("Sara", new int[]{95, 92, 98}),
            new Student("Ahmed", new int[]{70, 75, 80})
        };

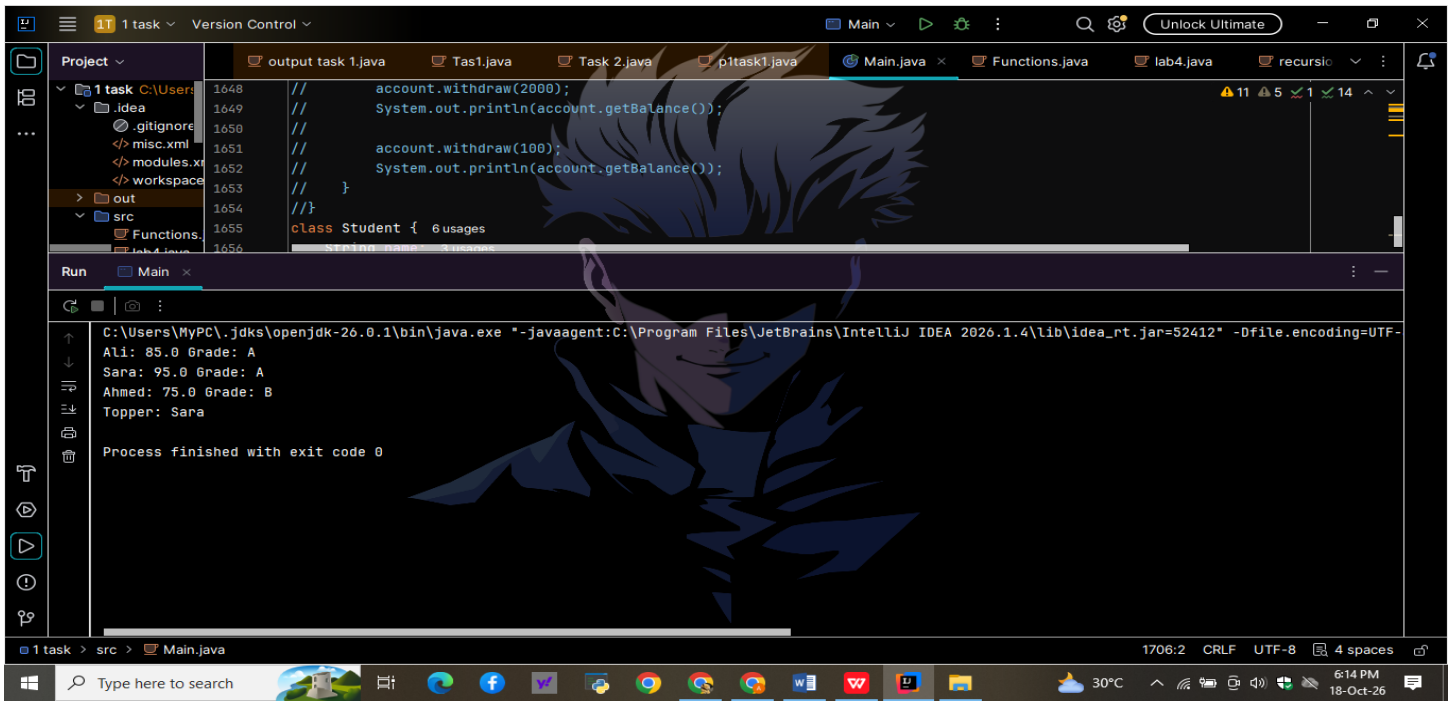
        Student topper = students[0];

        for (Student s : students) {
            System.out.println(s.name + ": " + s.average()
                    + " Grade: " + s.grade());

            if (s.average() > topper.average()) {
                topper = s;
            }
        }

        System.out.println("Topper: " + topper.name);
    }
}
```

Output:



Task 5: Create an abstract class Shape with an abstract method area(). Extend it with Circle and Triangle. Store mixed objects in a Shape array and print the total area using polymorphism.

```
abstract class Shape {
    abstract double area();
}

class Circle extends Shape {
    double radius;

    Circle(double radius) {
        this.radius = radius;
    }

    double area() {
        return Math.PI * radius * radius;
    }
}
```

```
class Triangle extends Shape {
    double base, height;

    Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }

    double area() {
        return 0.5 * base * height;
    }
}

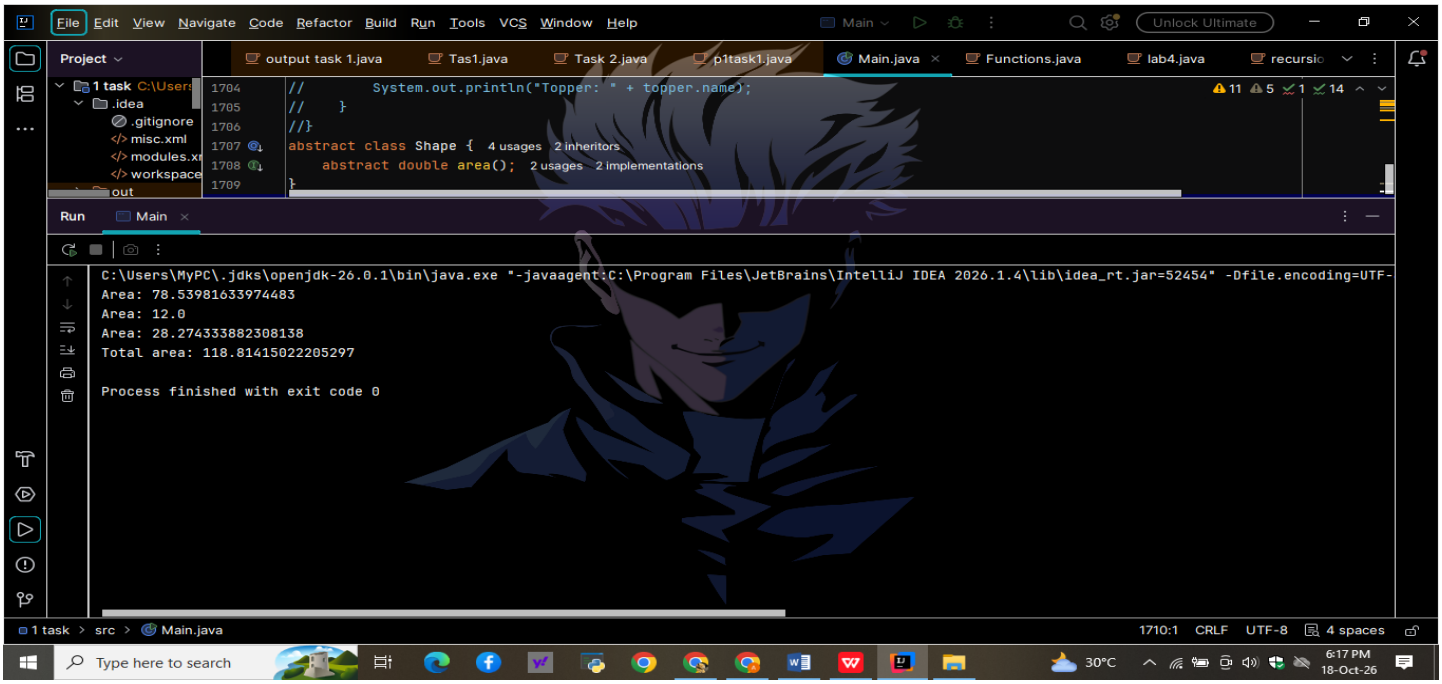
public class Main {
    public static void main(String[] args) {
        Shape[] shapes = {
            new Circle(5),
            new Triangle(4, 6),
            new Circle(3)
        };

        double total = 0;

        for (Shape s : shapes) {
            System.out.println("Area: " + s.area());
            total += s.area();
        }

        System.out.println("Total area: " + total);
    }
}
```

Output:


File Edit View Navigate Code Refactor Build Run Tools VCS Window Help
Main
Unlock Ultimate
Project
output task 1.java
Tas1.java
Task 2.java
p1task1.java
Main.java
Functions.java
lab4.java
1 task C:\Users
.idea
.gitignore
misc.xml
modules.xml
workspace
out
1704 // System.out.println("Topper: " + topper.name);
1705 // }
1706 //}
1707 abstract class Shape { 4 usages 2 inheritors
1708 abstract double area(); 2 usages 2 implementations
1709 }
Run
Main
C:\Users\MyPC\.jdks\openjdk-26.0.1\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.4\lib\idea_rt.jar=52454" -Dfile.encoding=UTF-
Area: 78.53981633974483
Area: 12.0
Area: 28.274333882308138
Total area: 118.81415022205297
Process finished with exit code 0
1 task > src > Main.java
1710:1 CRLF UTF-8 4 spaces
Type here to search
30°C
6:17 PM
18-Oct-26

Task 6:

Create a Point class with a method distanceTo(Point p). Read three points and decide whether they form a right-angled triangle.

```
class Point {
    double x, y;

    Point(double x, double y) {
        this.x = x;
        this.y = y;
    }

    double distanceTo(Point p) {
        double dx = x - p.x;
        double dy = y - p.y;

        return Math.sqrt(dx * dx + dy * dy);
    }
}

public class Main {
    public static void main(String[] args) {
        Point a = new Point(0, 0);
        Point b = new Point(3, 0);
        Point c = new Point(0, 4);

        double ab = a.distanceTo(b);
        double ac = a.distanceTo(c);
        double bc = b.distanceTo(c);

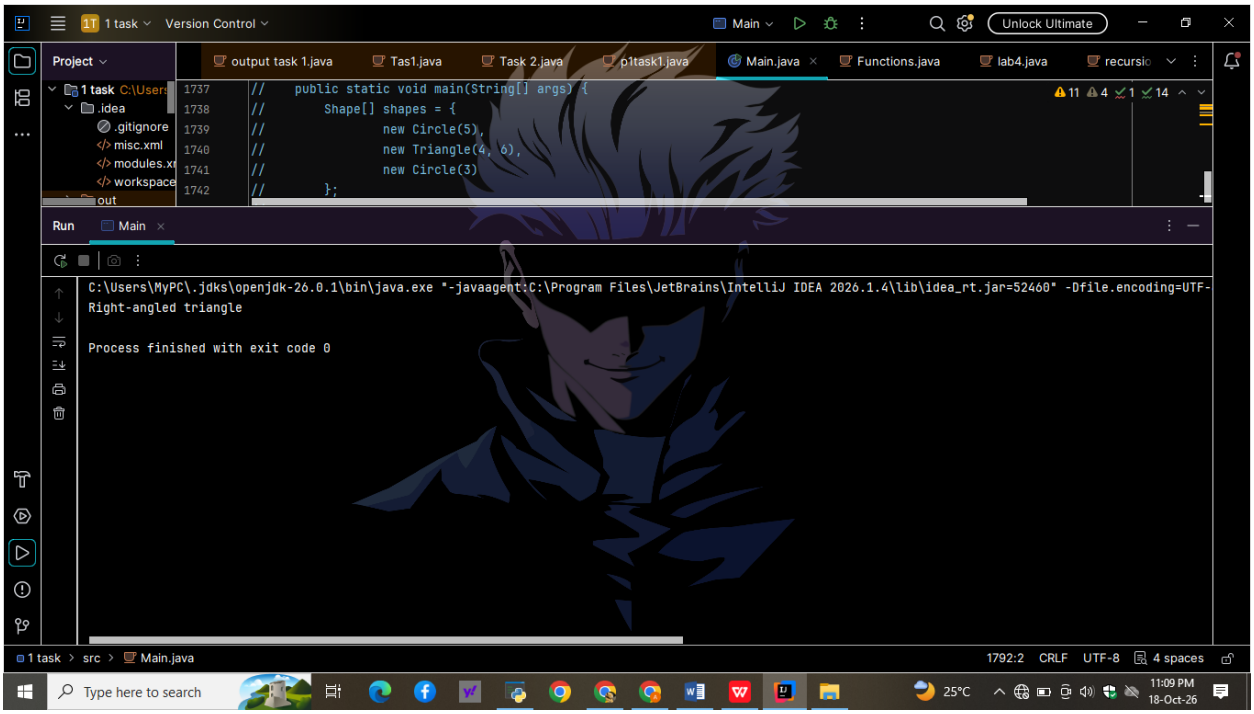
        double[] sides = {ab, ac, bc};

        java.util.Arrays.sort(sides);

        if (Math.abs(sides[0] * sides[0]
                + sides[1] * sides[1]
```

```
            - sides[2] * sides[2]) < 0.000001) {
            System.out.println("Right-angled triangle");
        } else {
            System.out.println("Not a right-angled triangle");
        }
    }
}
```

Output:



Task 7:

Create a Calculator class with overloaded add() methods for (int,int), (double,double), (int,int,int) and (String,String). Each method must print which version was called. Call them with mixed arguments such as add(5, 2.5), add('a', 1) and add("5", "2").

```
class Calculator {
    void add(int a, int b) {
        System.out.println("Integer version: " + (a + b));
    }

    void add(double a, double b) {
        System.out.println("Double version: " + (a + b));
    }
```

```
    void add(int a, int b, int c) {
        System.out.println("Three integer version: " + (a + b + c));
    }

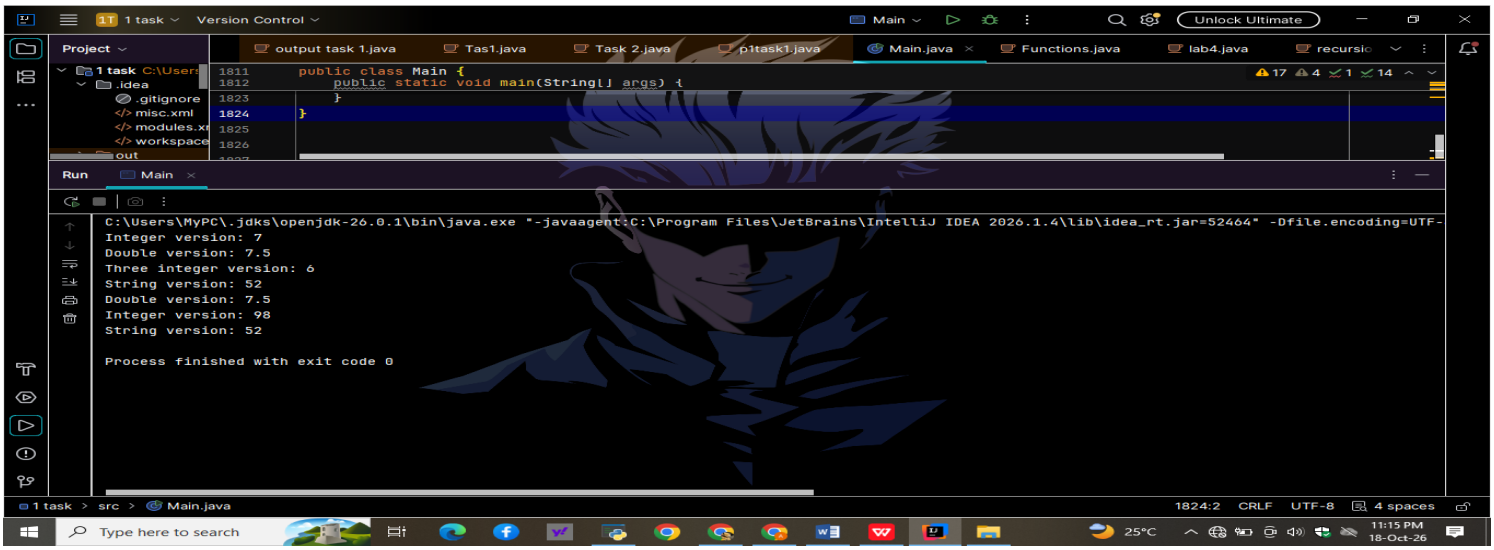
    void add(String a, String b) {
        System.out.println("String version: " + a + b);
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator c = new Calculator();

        c.add(5, 2);
        c.add(5.0, 2.5);
        c.add(1, 2, 3);
        c.add("5", "2");

        c.add(5, 2.5);
        c.add('a', 1);
        c.add("5", "2");
    }
}
```

Output:



Task 8:

Create an Employee class and a Manager subclass that overrides calculateSalary() and uses super. Store both types in one Employee array and print every salary.

```
class Employee {
    String name;
    double salary;

    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }

    double calculateSalary() {
        return salary;
    }
}

class Manager extends Employee {
    double bonus;

    Manager(String name, double salary, double bonus) {
        super(name, salary);
        this.bonus = bonus;
    }

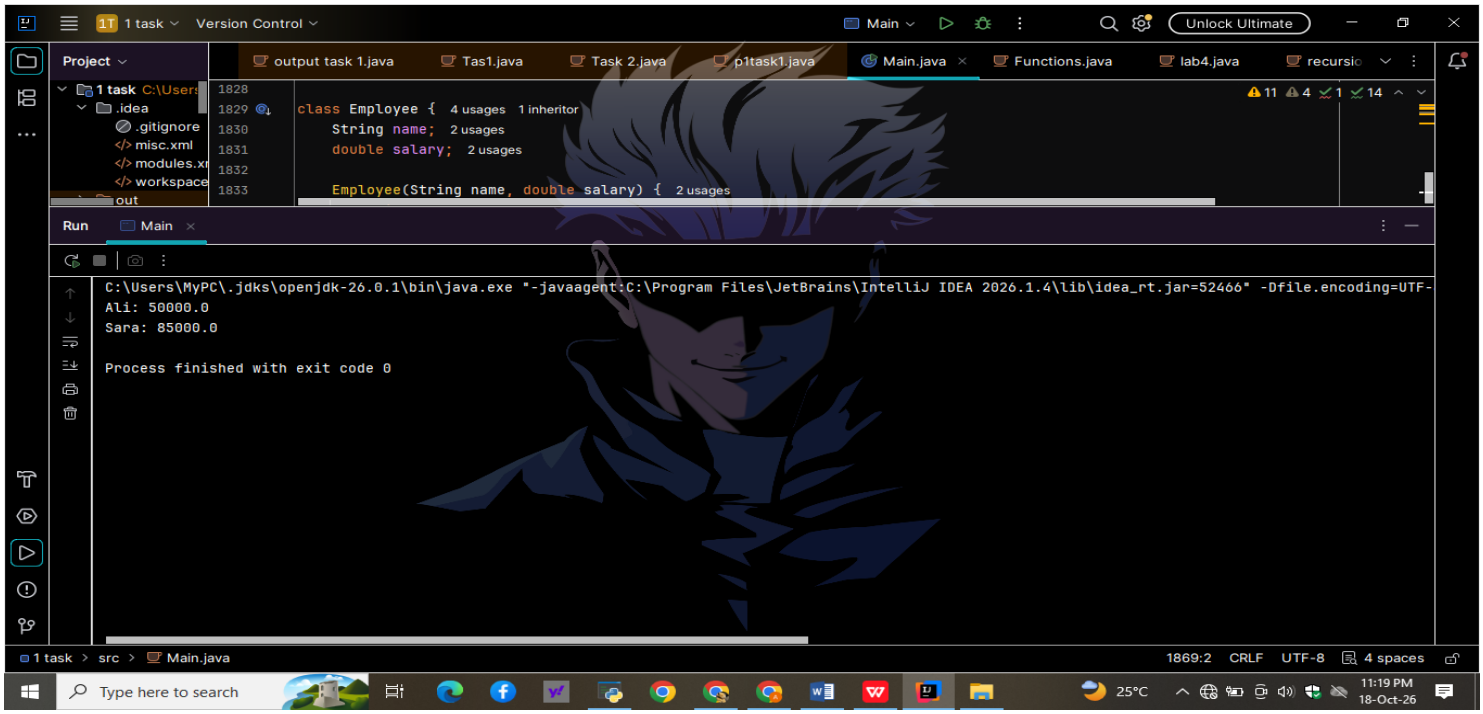
    @Override
    double calculateSalary() {
        return super.calculateSalary() + bonus;
    }
```

```
}

public class Main {
    public static void main(String[] args) {
        Employee[] employees = {
            new Employee("Ali", 50000),
            new Manager("Sara", 70000, 15000)
        };

        for (Employee e : employees) {
            System.out.println(e.name + ": "
                    + e.calculateSalary());
        }
    }
}
```

Output:



Task 9 : Build a Stack class using an array with push, pop, peek and isEmpty, handling overflow and underflow. Use it to check whether a String of (), {} and [] brackets is balanced.

```
class Stack {
    char[] data;
    int top = -1;

    Stack(int size) {
        data = new char[size];
    }

    void push(char value) {
        if (top == data.length - 1) {
            throw new RuntimeException("Stack overflow");
        }

        data[++top] = value;
    }

    char pop() {
        if (isEmpty()) {
            throw new RuntimeException("Stack underflow");
        }

        return data[top--];
    }

    char peek() {
        if (isEmpty()) {
            throw new RuntimeException("Stack is empty");
        }

        return data[top];
    }
```

```
    boolean isEmpty() {
        return top == -1;
    }
}

public class Main {
    static boolean isBalanced(String text) {
        Stack stack = new Stack(text.length());

        for (char ch : text.toCharArray()) {
            if (ch == '(' || ch == '{' || ch == '[') {
                stack.push(ch);
            } else if (ch == ')' || ch == '}' || ch == ']') {
                if (stack.isEmpty()) return false;

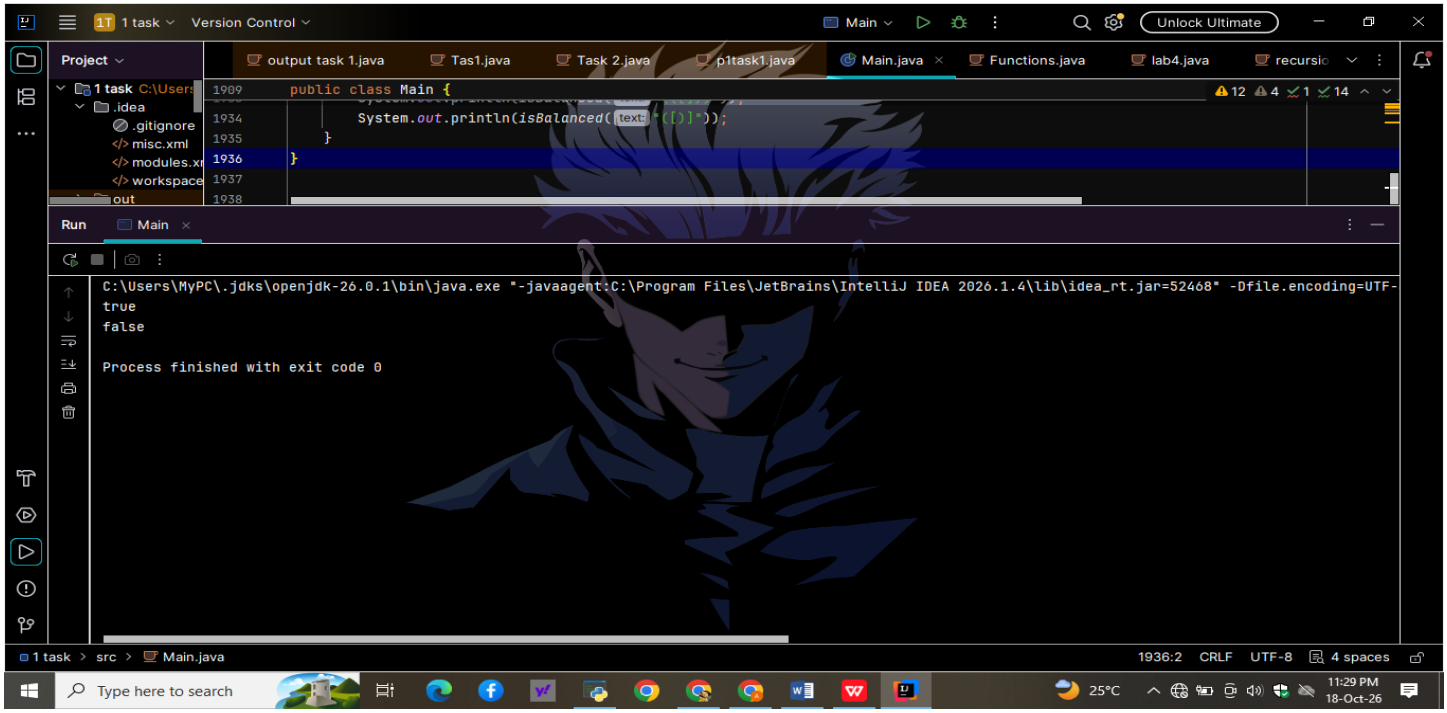
                char open = stack.pop();

                if ((ch == ')' && open != '(')
                        || (ch == '}' && open != '{')
                        || (ch == ']' && open != '[')) {
                    return false;
                }
            }
        }

        return stack.isEmpty();
    }

    public static void main(String[] args) {
        System.out.println(isBalanced("({[]})"));
        System.out.println(isBalanced("([)]"));
    }
}
```

Output:



Task 10:

Create class A and class B extends A. Each must have a static block, an instance block and a constructor that print a message. Run new B() twice and print the complete output.

```
class A {
   static {
      System.out.println("A static block");
   }

   {
      System.out.println("A instance block");
   }

   A() {
      System.out.println("A constructor");
   }
}
```

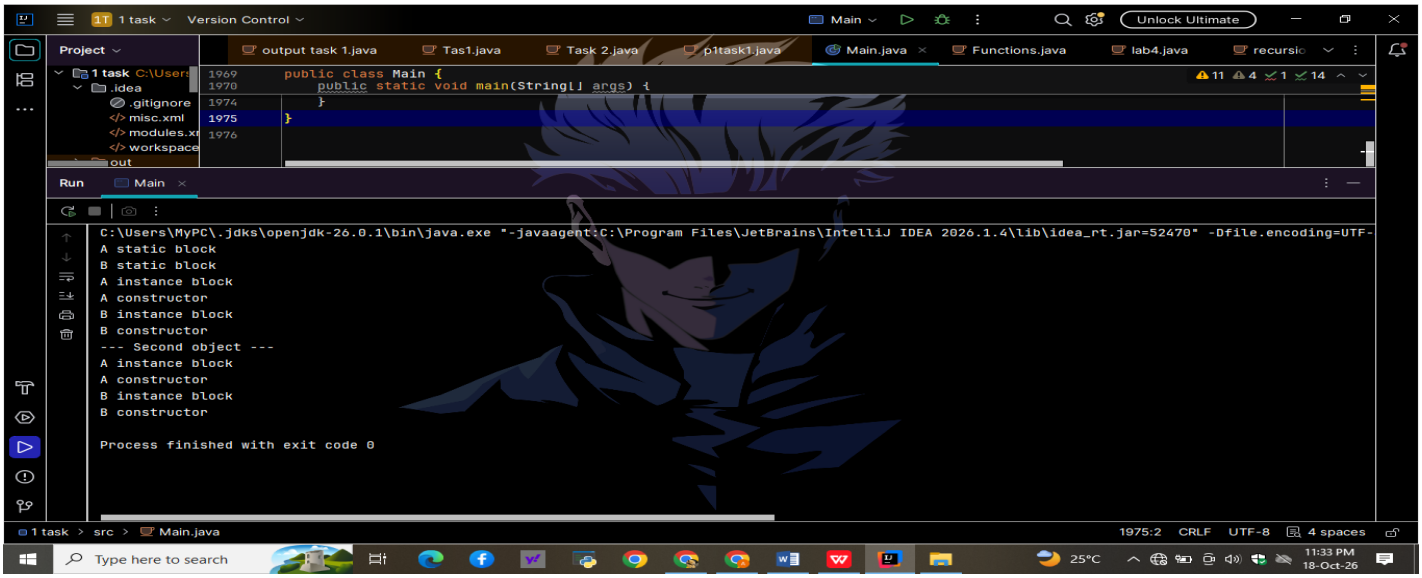
```
class B extends A {
    static {
        System.out.println("B static block");
    }

    {
        System.out.println("B instance block");
    }

    B() {
        System.out.println("B constructor");
    }
}

public class Main {
    public static void main(String[] args) {
        new B();
        System.out.println("--- Second object ---");
        new B();
    }
}
```

Output:



Task 11:

Create an immutable class Money (final fields, no setters) with add(Money) returning a new object. Override equals() and hashCode(), and demonstrate the difference between == and equals().

```
import java.util.Objects;


final class Money {
    private final int amount;

    Money(int amount) {
        this.amount = amount;
    }

    Money add(Money other) {
        return new Money(this.amount + other.amount);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Money)) return false;

        Money other = (Money) obj;
```

```
        return amount == other.amount;
    }

    @Override
    public int hashCode() {
        return Objects.hash(amount);
    }

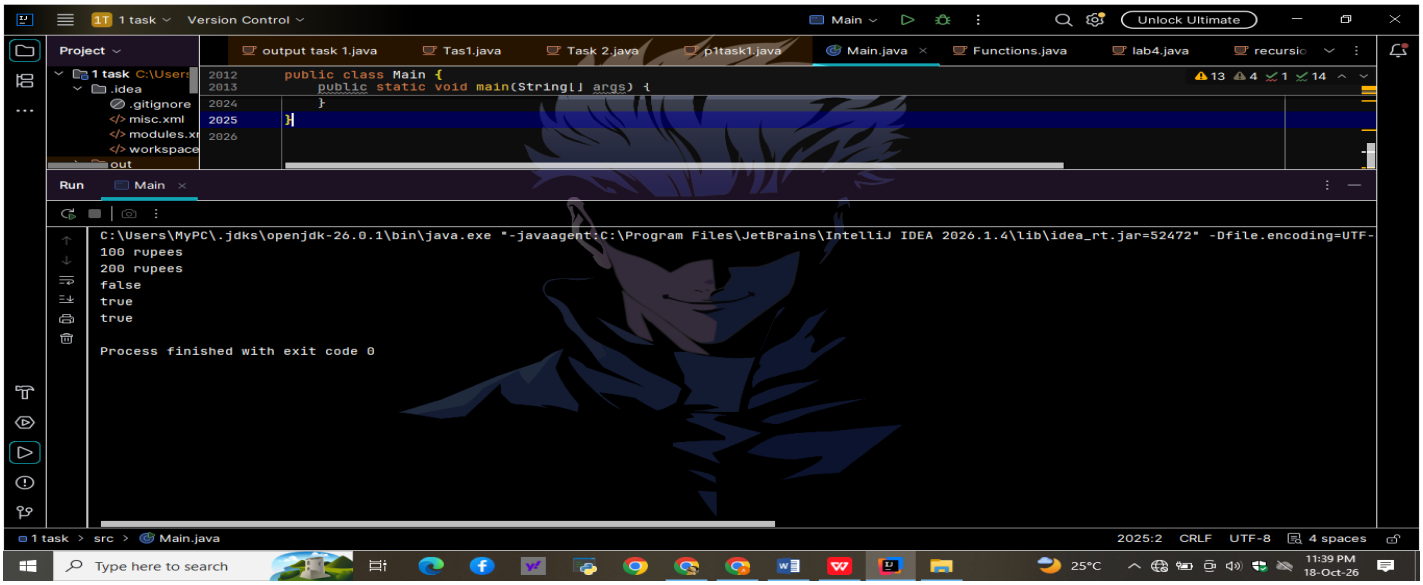
    @Override
    public String toString() {
        return amount + " rupees";
    }
}

public class Main {
    public static void main(String[] args) {
        Money m1 = new Money(100);
        Money m2 = new Money(100);
        Money m3 = m1.add(m2);

        System.out.println(m1);
        System.out.println(m3);
```

```
        System.out.println(m1 == m2);
        System.out.println(m1.equals(m2));
        System.out.println(m1.hashCode() == m2.hashCode());
    }
}
```

Output:



Task 12:

Implement a singly linked list from scratch (Node class plus LinkedList class) with insertAtEnd, deleteByValue and an iterative reverse. Print the list before and after reversing.

```
class LinkedList {
    class Node {
        int data;
        Node next;

        Node(int data) {
            this.data = data;
        }
    }
```

```
Node head;

void insertAtEnd(int value) {
   Node newNode = new Node(value);

   if (head == null) {
      head = newNode;
      return;
   }

   Node current = head;

   while (current.next != null) {
      current = current.next;
   }

   current.next = newNode;
}

void deleteByValue(int value) {
   if (head == null) return;

   if (head.data == value) {
      head = head.next;
      return;
   }

   Node current = head;

   while (current.next != null
         && current.next.data != value) {
      current = current.next;
   }
```

```
        if (current.next != null) {
            current.next = current.next.next;
        }
    }

    void reverse() {
        Node previous = null;
        Node current = head;

        while (current != null) {
            Node nextNode = current.next;
            current.next = previous;
            previous = current;
            current = nextNode;
        }

        head = previous;
    }

    void display() {
        Node current = head;

        while (current != null) {
            System.out.print(current.data + " -> ");
            current = current.next;
        }

        System.out.println("null");
    }
}

public class Main {
    public static void main(String[] args) {
```

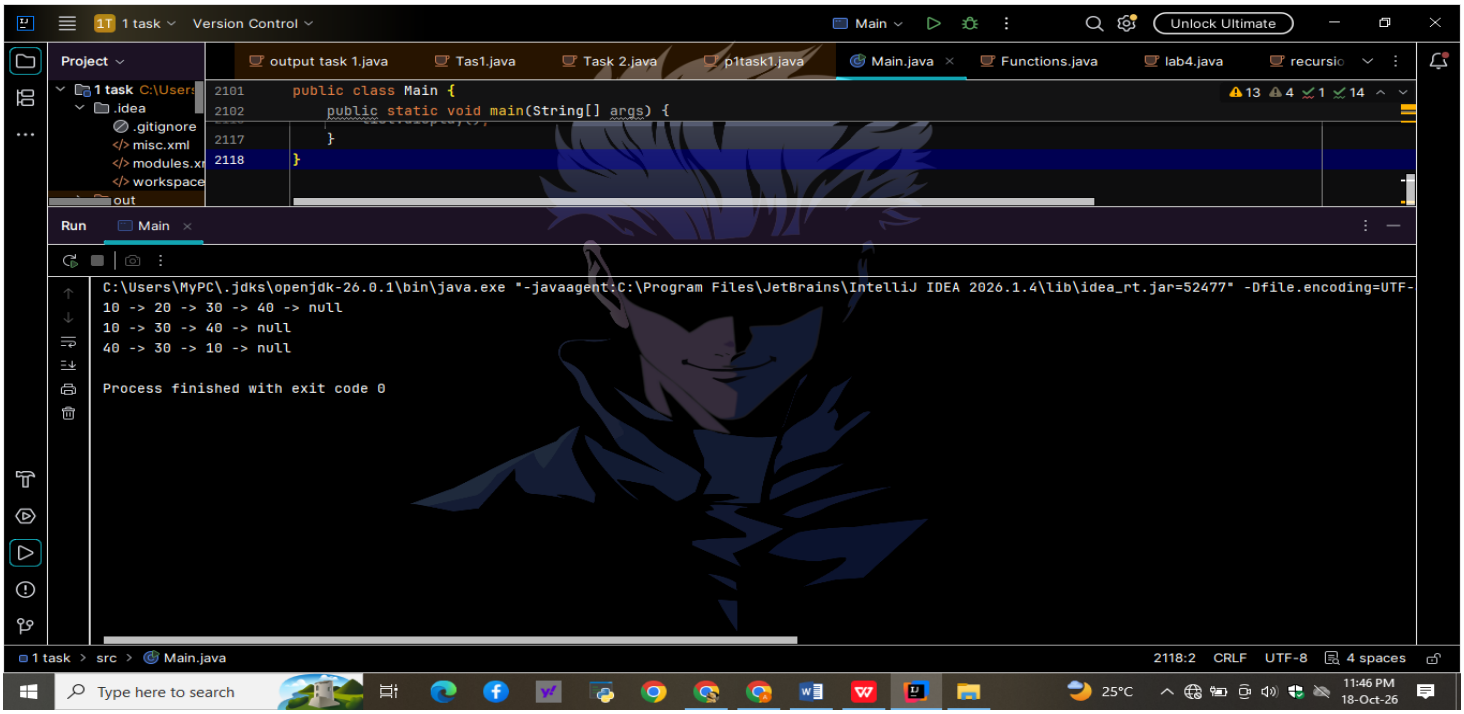
```
        LinkedList list = new LinkedList();

        list.insertAtEnd(10);
        list.insertAtEnd(20);
        list.insertAtEnd(30);
        list.insertAtEnd(40);

        list.display();

        list.deleteByValue(20);
        list.display();

        list.reverse();
        list.display();
    }
}
```



Task 13: Create a checked custom exception InsufficientFundsException and use it in a withdraw() method. Show a case where a finally block runs even though the try block executes a return statement.

```
class InsufficientFundsException extends Exception {
    InsufficientFundsException(String message) {
        super(message);
    }
}

class BankAccount {
    private double balance;

    BankAccount(double balance) {
        this.balance = balance;
    }

    void withdraw(double amount)
            throws InsufficientFundsException {
        if (amount > balance) {
            throw new InsufficientFundsException(
                    "Insufficient funds");
        }

        balance -= amount;
        System.out.println("Remaining balance: " + balance);
    }
}

public class Main {
```

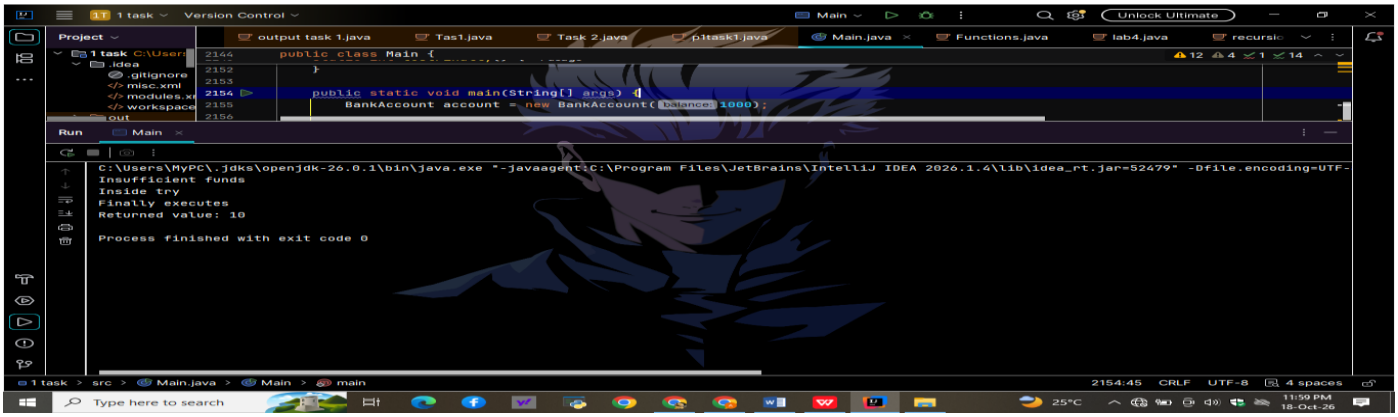
```
    static int testFinally() {
        try {
            System.out.println("Inside try");
            return 10;
        } finally {
            System.out.println("Finally executes");
        }
    }

    public static void main(String[] args) {
        BankAccount account = new BankAccount(1000);

        try {
            account.withdraw(1500);
        } catch (InsufficientFundsException e) {
            System.out.println(e.getMessage());
        }

        System.out.println("Returned value: " + testFinally());
    }
}
```

Output:



Task 14: Implement a Singleton class with lazy initialization. Prove that only one instance exists by calling it 5 times, then make it thread-safe.

```
class Singleton {
    private static volatile Singleton instance;

    private Singleton() {
        System.out.println("Object created");
    }

    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }

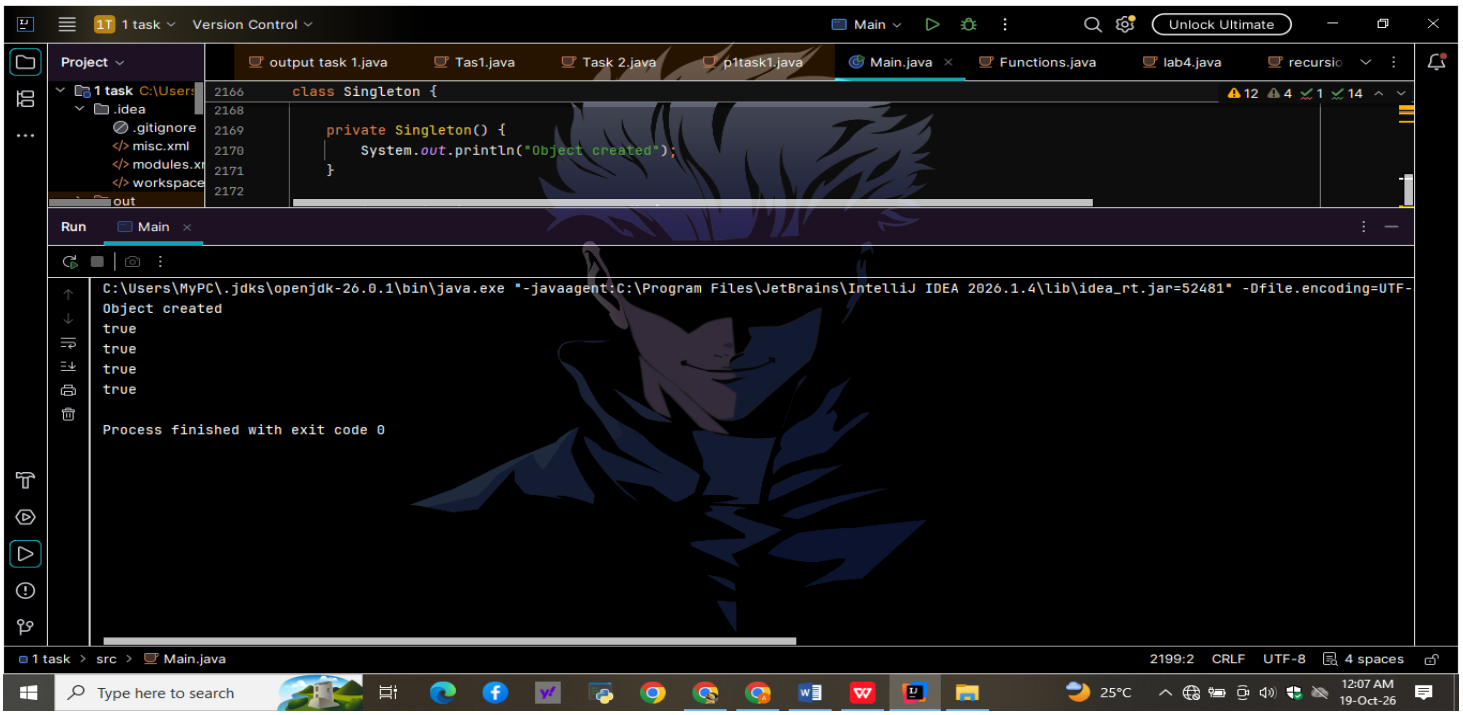
        return instance;
    }
}

public class Main {
    public static void main(String[] args) {
        Singleton a = Singleton.getInstance();
```

```
        Singleton b = Singleton.getInstance();
        Singleton c = Singleton.getInstance();
        Singleton d = Singleton.getInstance();
        Singleton e = Singleton.getInstance();

        System.out.println(a == b);
        System.out.println(b == c);
        System.out.println(c == d);
        System.out.println(d == e);
    }
}
```

Output:



Task 15:

Create two interfaces A and B that both have a default method hello(). Write a class C that implements both, resolves the conflict, and calls each interface's version.

```
interface A {
    default void hello() {
        System.out.println("Hello from A");
    }
}

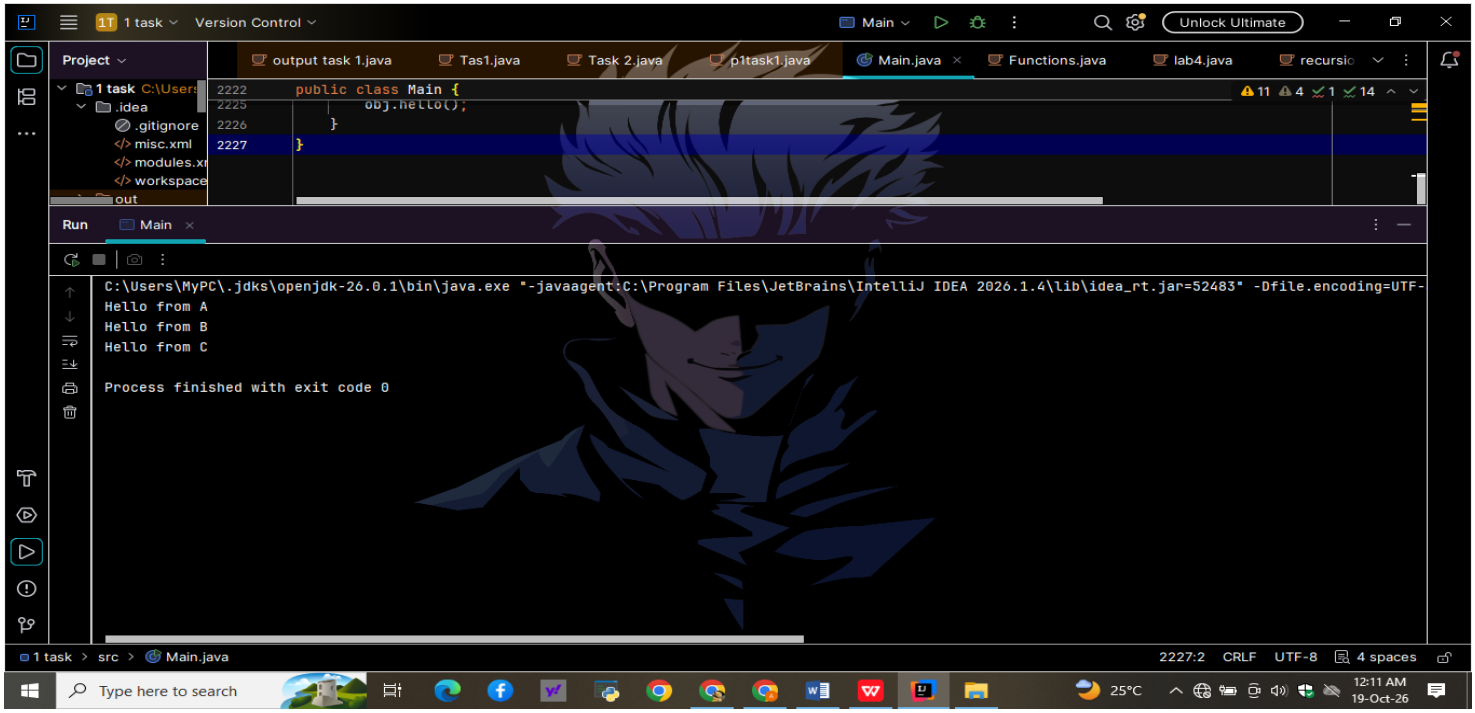
interface B {
    default void hello() {
        System.out.println("Hello from B");
    }
}

class C implements A, B {
    @Override
    public void hello() {
        A.super.hello();
        B.super.hello();

        System.out.println("Hello from C");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        C obj = new C();
        obj.hello();
    }
}
```

Output:



Task 16: Create a generic class Pair<K, V> and a generic method max() that works on any array of Comparable objects. Test it with Integer, String and a custom Student class that implements Comparable.

```
class Pair<K, V> {
    K key;
    V value;

    Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    void display() {
        System.out.println(key + " : " + value);
    }
}

class Student implements Comparable<Student> {
    String name;
    int marks;

    Student(String name, int marks) {
        this.name = name;
        this.marks = marks;
    }

    @Override
    public int compareTo(Student other) {
        return Integer.compare(this.marks, other.marks);
    }

    @Override
    public String toString() {
        return name + " (" + marks + ")";
    }
}

public class Main {
    static <T extends Comparable<T>> T max(T[] array) {
```

```
        T largest = array[0];

        for (T item : array) {
            if (item.compareTo(largest) > 0) {
                largest = item;
            }
        }

        return largest;
    }

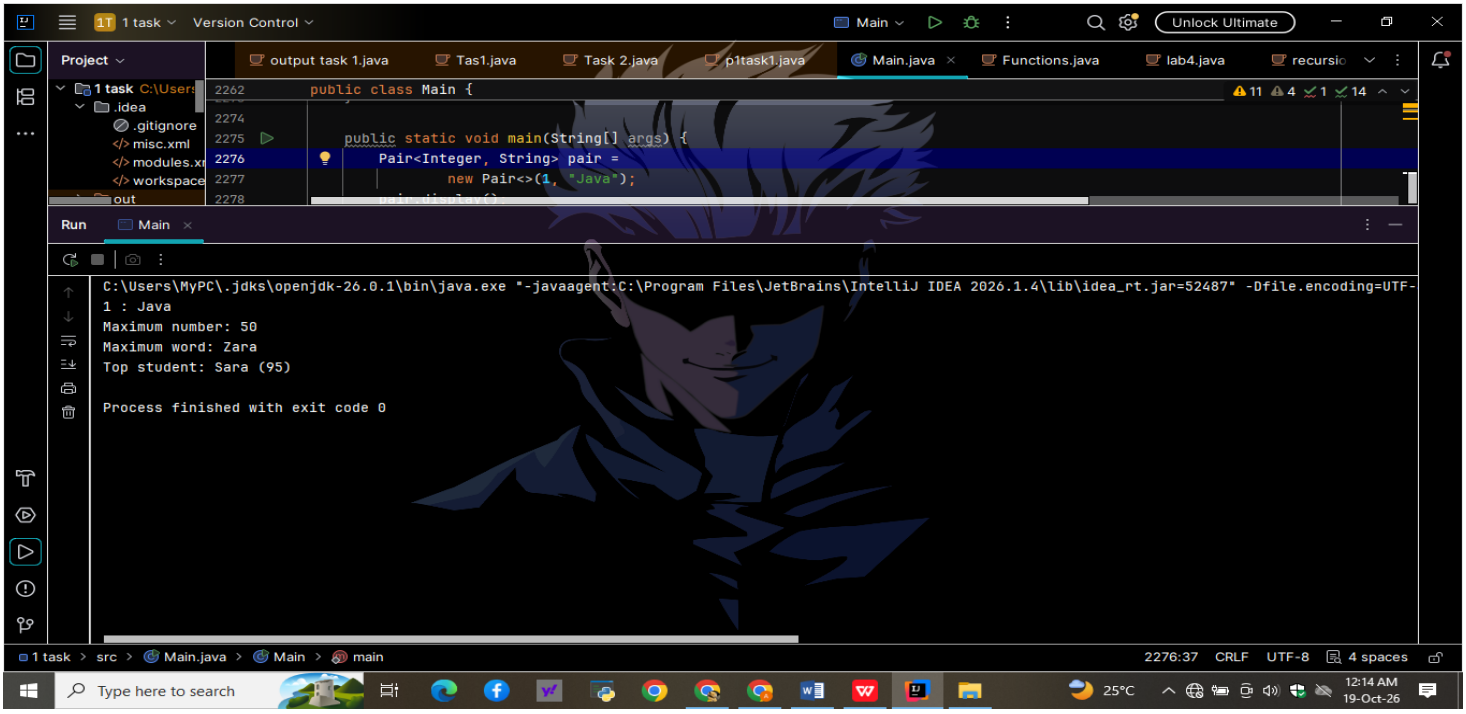
    public static void main(String[] args) {
        Pair<Integer, String> pair =
                new Pair<>(1, "Java");
        pair.display();

        Integer[] numbers = {10, 50, 30};
        String[] words = {"Ali", "Zara", "Ahmed"};

        Student[] students = {
            new Student("Ali", 75),
            new Student("Sara", 95),
            new Student("Ahmed", 80)
        };

        System.out.println("Maximum number: " + max(numbers));
        System.out.println("Maximum word: " + max(words));
        System.out.println("Top student: " + max(students));
    }
}
```

Output:



Task 17:

Build a mini library system with Book, Member and Library classes. A member can borrow at most 3 books, and an already issued book cannot be borrowed again. Simulate borrow and return operations and print the status after each.

```
import java.util.ArrayList;


class Book {
   String title;
   boolean issued = false;


   Book(String title) {
      this.title = title;
   }
}
```

```
class Member {
    String name;
    ArrayList<Book> borrowed = new ArrayList<>();

    Member(String name) {
        this.name = name;
    }

    boolean borrow(Book book) {
        if (borrowed.size() >= 3) {
            System.out.println(name + " already has 3 books");
            return false;
        }

        if (book.issued) {
            System.out.println(book.title + " is already issued");
            return false;
        }

        book.issued = true;
        borrowed.add(book);

        System.out.println(name + " borrowed " + book.title);
        return true;
    }

    void returnBook(Book book) {
        if (borrowed.remove(book)) {
            book.issued = false;
            System.out.println(name + " returned " + book.title);
        } else {
            System.out.println(name + " does not have this book");
        }
```

```
    }
}

class Library {
    ArrayList<Book> books = new ArrayList<>();
    ArrayList<Member> members = new ArrayList<>();

    void addBook(Book book) {
        books.add(book);
    }

    void addMember(Member member) {
        members.add(member);
    }

    void displayStatus() {
        System.out.println("\nLibrary status:");

        for (Book book : books) {
            System.out.println(book.title + " - "
                    + (book.issued ? "Issued" : "Available"));
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Library library = new Library();

        Book b1 = new Book("Java");
        Book b2 = new Book("Python");
        Book b3 = new Book("Database");
        Book b4 = new Book("Networking");
```

```
        library.addBook(b1);
        library.addBook(b2);
        library.addBook(b3);
        library.addBook(b4);

        Member m1 = new Member("Ali");
        Member m2 = new Member("Sara");

        library.addMember(m1);
        library.addMember(m2);

        m1.borrow(b1);
        library.displayStatus();

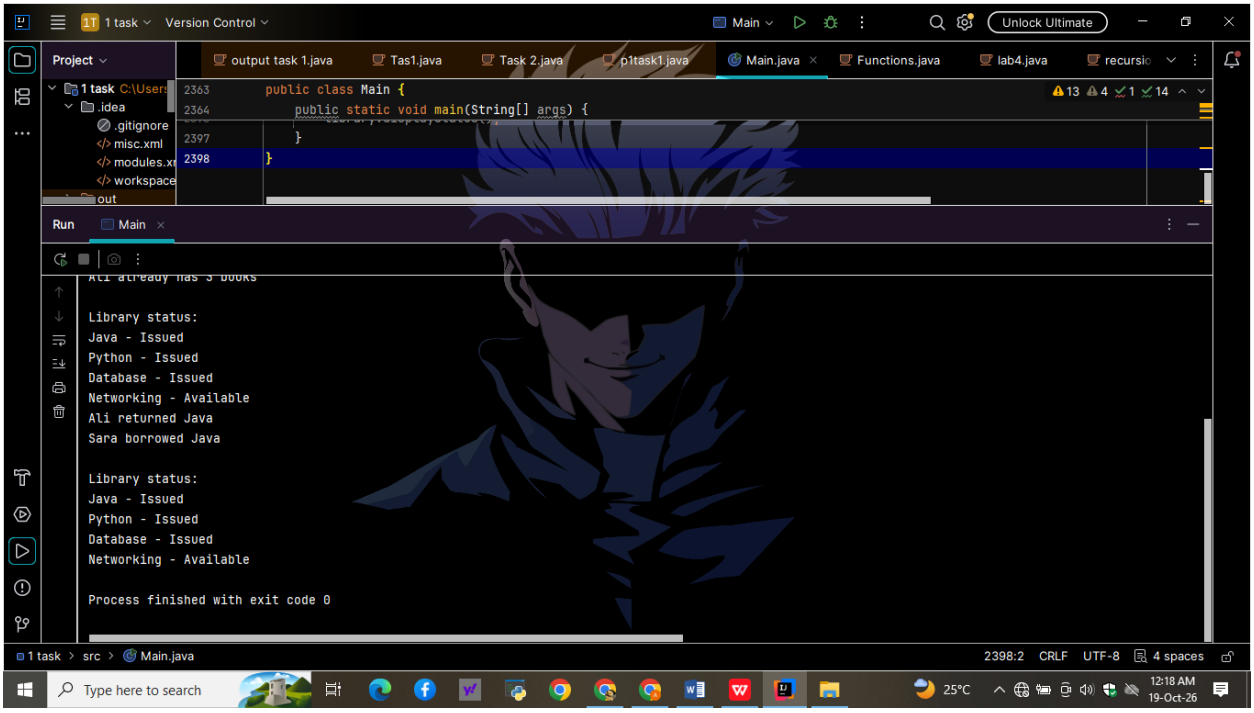
        m2.borrow(b1); // Already issued
        m1.borrow(b2);
        m1.borrow(b3);
        m1.borrow(b4); // Maximum limit reached

        library.displayStatus();

        m1.returnBook(b1);
        m2.borrow(b1);

        library.displayStatus();
    }
}
```

Output:


Library status:
Java - Issued
Python - Issued
Database - Issued
Networking - Available
Ali returned Java
Sara borrowed Java
Library status:
Java - Issued
Python - Issued
Database - Issued
Networking - Available
Process finished with exit code 0